Creational patterns

Factory method

Why? Because we currently have two abstract classes (SlideItem and Accessor) that have more than one concrete class. Using the factory method, we can ensure that if we ever add more classes stemming from these abstract classes, we won't have to change the entire codebase again and again.

Why not use Abstract factory instead? Simply because we feel it would be overkill for this application. The advantage of using Abstract factory over Factory method is that it can create different versions of products. For example, instead of a colorful BitmapItem, we can create a black and white one. Right now, the only classes we could use Abstract factory on are SlideItem and Accessor and none of them are in a "family" of products. Because there is nothing like this in Jabberpoint currently, implementing Abstract factory would just overcomplicate the code, and besides we can always evolve the Factory method into Abstract factory later in the application's life.

Prototype vs. Singleton or Builder

Why? Because we have the Slide class currently, there are usually multiple instances of this class within the Presentation class, and so it would be useful to be able to create them more easily, for example, making a template for the cloning of blank classes.

Why not Singleton or Builder?

- Singleton ensures only one instance of a class exists. JabberPoint needs multiple slides and items, so limiting instances would make no sense.
- Builder would help construct complex objects, but slides and elements are already self-contained—we just need to clone them, not build them step-by-step.

Structural Patterns

Composite vs. Bridge or Flyweight

Why Composite?
JabberPoint's presentation model is hierarchical: A Presentation contains multiple Slides. A Slide contains multiple SlideItems (text, images, etc.). Composite lets us treat individual elements and groups of elements uniformly, simplifying rendering, iteration, and modifications. A draw() method, for example, works the same for a single text item or a whole slide.

Why not Bridge or Flyweight?

- Bridge is useful when you need to separate abstraction from implementation, like supporting different rendering backends. But JabberPoint only has one rendering system, so Bridge is unnecessary.
- Flyweight optimizes memory usage when many objects share the same data (e.g., repeated icons). Since slides and elements are usually unique, there's little to gain from Flyweight.

Trade-offs: Composite makes the structure clear but requires defining a common interface (SlideComponent). This small effort is worth it for the clarity and flexibility it provides.

Decorator vs. Proxy or Adapter

Why Decorator?

JabberPoint may need optional features for slides, such as adding slide numbers, watermarks, or highlights. Instead of modifying Slide directly, Decorator lets us wrap a slide or item dynamically to add behavior without altering existing classes.

Why not Proxy or Adapter?

- Proxy controls access or defers loading (e.g., lazy loading images), which is useful for optimizing performance but doesn't help add features dynamically.
- Adapter makes an interface compatible with another system, which is great for integrating third-party slide elements, but JabberPoint already controls its internal format.

Trade-offs: The only downside is that Decorators increase the number of small classes, but that's manageable. Adding features dynamically is worth it.

Behavioral Patterns

Command vs. Strategy or Observer

Why Command?

JabberPoint executes user actions dynamically—things like "Next Slide", "Undo", "Open File". Instead of hardcoding UI buttons to directly modify slides, Command encapsulates actions into objects, making it easy to queue, log, and undo actions.

Why not Strategy or Observer?

- Strategy is for selecting an algorithm dynamically (e.g., choosing different sorting methods). JabberPoint's user actions aren't algorithms—they're specific operations that need to be executed, so Strategy doesn't fit.
- Observer notifies multiple components when something changes (e.g., updating thumbnails when a slide changes). While useful for notifications, it doesn't encapsulate user actions like Command does.

Trade-offs: More classes are needed (one per command), but this is a small price for clear separation between UI and business logic.

Memento vs. State or Snapshot

Why Memento?

JabberPoint needs undo/redo functionality, which means capturing and restoring the exact state of slides and presentations. Memento lets us save snapshots of an object's state without exposing its internals, allowing clean rollback functionality.

Why not State or Snapshot?

- State is for changing an object's behavior based on its internal state (like a media player switching between play, pause, and stop). JabberPoint's undo/redo doesn't change behavior dynamically—it restores previous states, making State the wrong tool.
- Snapshot is a general term for saving object state, and Memento is a formalized version of it that keeps encapsulation intact, making it the better choice.

Trade-offs: Memento increases memory usage (saving many snapshots), but we mitigate this by limiting undo levels.